Fundamentals of Natural Language Processing

Chapter 2 — Text Representation and Evaluation

Aymen Ben Brik

Chapter 2

Text Representation and Evaluation

A computer cannot operate on raw text. Before any model can "read" a sentence, the sentence must be *tokenized* (cut into discrete units), *numericalized* (converted into integer indices), and *vectorized* (embedded in a vector space). This chapter develops the chain rigorously: tokenization granularities, sparse and dense vector-space models, semantic similarity, and the evaluation metrics by which we compare NLP systems.

2.1 Tokenization fundamentals

2.1.1 Motivation

Tokenization is the unsung hero of every NLP pipeline. A poor tokenizer silently undermines the most sophisticated downstream model: out-of-vocabulary words become unknown tokens, morphological variants are treated as unrelated, multi-word expressions are fragmented. Conversely, a tokenizer aligned with the data distribution can shrink vocabulary size, accelerate training, and improve generalisation. Modern Large Language Models all rely on a single technical choice — *sub-word tokenization* — whose strengths and limitations every practitioner should understand.

2.1.2 Approach by problem: how many tokens are in this sentence?

Problem. Consider the sentence

> *"I've been to Sfax-University; it's amazing!"*

Count how many "tokens" it contains under each of the following naive policies:

1. split on whitespace;
2. split on whitespace and punctuation;
3. split into individual characters.

Solution sketch.

1. Whitespace only: {`I've`, `been`, `to`, `Sfax-University;`, `it's`, `amazing!`} — 6 tokens. Punctuation glued to words; "I've" is a single token.

2. Whitespace and punctuation: {`I`, `'ve`, `been`, `to`, `Sfax`, `-`, `University`, `;`, `it`, `'s`, `amazing`, `!`} — 12 tokens. The contraction *I've* is split, but so is the legitimate compound *Sfax-University*.

3. Characters: 35 tokens. Punctuation, capitalisation and spaces are all preserved, but every word is fragmented; the model would have to relearn "what is a word" from data.

Each policy entails a trade-off between vocabulary size and information preserved. There is no universally correct answer; the choice is engineering, guided by the downstream task and the language being modelled.

2.1.3 Definition and the tokenization pipeline

Definition 2.1.1 (Tokenization)**. Tokenization** is the process of converting a raw text string into an ordered sequence of discrete units called **tokens**, drawn from a finite inventory called the **vocabulary** V.

Definition 2.1.2 (Numericalization)**. Numericalization** is the bijective mapping $V \leftrightarrow \{0, 1, \ldots, |V| - 1\}$ that assigns each token an integer identifier.

The full pipeline is: *raw text* $\rightarrow$ tokens $\rightarrow$ token IDs $\rightarrow$ tensor inputs to a model. Every NLP system is built on top of these three trivially simple but practically critical steps.

2.1.4 Three levels of granularity

Level	Vocabulary size	Sequence length	OOV behaviour
Word	Very large (10^5–10^6)	Short	Unknown words become `[UNK]`
Character	Tiny (~ 100)	Very long	No OOV (any character is in V)
Sub-word	Moderate (10^4–10^5)	Moderate	Robust: any word splits into known pieces

Word tokenization. The intuitive default. Tokens are entire words separated by whitespace and punctuation rules. Strengths: each token carries lexical meaning, and downstream models converge faster on a small annotated dataset. Weaknesses: vocabulary explodes for morphologically rich languages (*Arabic, Turkish, Finnish*); typos and rare words become `[UNK]`; cannot generalise across morphological variants (*run / runs / running* are three unrelated tokens).

Character tokenization. Each character is a token. Strengths: vocabulary is bounded and stable across languages; no out-of-vocabulary problem (any unseen word becomes a sequence of known characters). Weaknesses: sequences become 5–10$\times$ longer, increasing both memory and compute; the model must learn from scratch which character sequences form meaningful units.

Sub-word tokenization. The dominant choice in 2020s NLP. Vocabulary contains *frequent words as wholes* and *rare words split into meaningful pieces*. The split is learnt from data using algorithms such as Byte-Pair Encoding (BPE), WordPiece (BERT) and SentencePiece (XLM-R, T5). Sub-word tokenization combines the strengths of word-level (semantic units for common words) and character-level (no OOV) approaches.

2.1.5 Pre-processing operations

Tokenization is often combined with ad-hoc transformations that modify or normalise the input string before splitting.

- **Lowercasing**: "Apple" and "apple" become equivalent. Useful for spelling-insensitive tasks but destroys signal in NER (a capitalised *Apple* is more likely the company).
- **Punctuation removal**: simplifies vocabulary but loses sentence boundaries.
- **Stop-word removal**: discard high-frequency words (*the, of, in*) that carry little task-specific information. Useful for keyword search; harmful for syntax-sensitive tasks.
- **Stemming**: chop morphological suffixes by hand-coded rules (Porter, Snowball). *running* $\to$ *run*. Fast but linguistically crude.
- **Lemmatization**: map a word to its dictionary form using a morphological analyser. *better* $\to$ *good*. Slower but linguistically principled.

Remark 2.1.1. Modern large language models perform little or no manual pre-processing: they are trained on raw text with sub-word tokenization, and the model itself learns whatever normalisations it needs. Heavy pre-processing is typical of statistical-era pipelines and remains useful for small-data, classical-ML systems.

2.1.6 Byte-Pair Encoding (BPE) in detail

BPE is the simplest sub-word algorithm and the workhorse of GPT-2, GPT-3, GPT-4, RoBERTa, and many others. It iteratively merges the most frequent pair of adjacent symbols.

Algorithm.

1. Start with a vocabulary that contains every individual character of the corpus.
2. Treat each word as a sequence of characters with an end-of-word marker.
3. Count the frequency of every adjacent symbol pair across the corpus.
4. Merge the most frequent pair into a new symbol; add it to the vocabulary.
5. Repeat steps 3–4 until a target vocabulary size is reached.

Example 2.1.1 (A miniature BPE run)**.** Suppose the corpus consists of the words `low`, `lower`, `newest`, `widest`, with frequencies $5, 2, 6, 3$. The initial vocabulary is {`l, o, w, e, r, n, s, t, i, d, </w>`}. Counting pairs:

$$\texttt{e s} = 6 + 3 = 9, \quad \texttt{s t} = 9, \quad \texttt{l o} = 7, \quad \texttt{o w} = 7, \ldots$$

The pair `e s` (9 occurrences) is merged first into a new symbol `es`. Recount, merge again — now `es t` (9 occurrences) into `est`. Continue: `lo`, `low`, `ne`, `new`, … After enough merges, frequent words like `low</w>` become single tokens, while rare words remain decomposed. A new word like `lowest` is then tokenized as `low + est`, even though it never appeared in training.

Why BPE works. The algorithm provides a natural rate–distortion trade-off. As vocabulary grows, the average sequence length shrinks, but the "unit cost" per token increases (each token carries more information). Empirically, vocabularies of 30k–100k sub-words give an excellent trade-off across many languages.

2.1.7 Multilingual and dialectal challenges

Tokenization choices that are adequate for English may fail for other languages.

- **Whitespace-free scripts.** Chinese, Japanese, Thai write without spaces. Word tokenization requires segmentation algorithms (e.g. Jieba, MeCab).
- **Morphologically rich languages.** Arabic and Turkish concatenate prefixes and suffixes onto a stem, generating very large surface-form inventories. Sub-word tokenization is essential.
- **Code-switching and dialects.** Tunisian Arabic frequently mixes Modern Standard Arabic, French, and Latin-script transliterations ("arabizi"). A general-purpose multilingual tokenizer (e.g. XLM-R's SentencePiece) is recommended over single-language tokenizers.
- **Right-to-left scripts.** Display order and logical order can differ; numerical IDs operate on logical order, but evaluation tools must be aware of the visual rendering.

2.1.8 Worked examples

Example 1 (Granularity choice — Easy). You are training a classifier on 1000 short SMS messages, mostly in English with frequent typos. Which tokenization granularity is most appropriate, and why?

Solution. Sub-word tokenization. Word-level would suffer from OOV (typos produce never-seen surface forms); character-level would lengthen sequences and dilute the signal in a small dataset. Sub-word splits typos into recognised pieces (`hapy` → `ha + py`) without exploding the vocabulary.

Example 2 (BPE iteration — Medium). Apply two iterations of BPE to the toy corpus {`aaab`, `aabb`, `abab`} (each of frequency 1). Show the resulting vocabulary after each merge.

Solution. Initial vocabulary: {`a`, `b`}.

Iteration 1: count pairs — `a a` appears in `aaab` (twice) and `aabb` (once) and `abab` (zero) = 3; `a b` appears in `aaab` (once) and `aabb` (once) and `abab` (twice) = 4; `b a` appears once (in `abab`); `b b` once. Most frequent: `a b` (4). Merge into `ab`. Vocabulary: {`a`, `b`, `ab`}. Re-tokenization: `aaab` → `a a ab`; `aabb` → `a ab b`; `abab` → `ab ab`.

Iteration 2: count new pairs — `a a` (in `aaab`, once) = 1; `a ab` (in `aaab` and `aabb`) = 2; `ab b` (in `aabb`) = 1; `ab ab` (in `abab`) = 1. Most frequent: `a ab` (2). Merge into `aab`. Vocabulary: {`a`, `b`, `ab`, `aab`}.

Example 3 (Tokenizer mismatch — Hard). A pretrained English BERT tokenizer applied to Tunisian Arabic in Latin script ("arabizi") splits the word `n7ebek` ("I love you") as `[n, ##7, ##e, ##bek]`. Discuss the consequences for a downstream classifier and propose two remediation strategies.

Solution.

- **Consequences.** The numeric digit 7 (which substitutes for the Arabic letter ʿ) is treated as foreign; the model sees an unfamiliar pattern with no semantic anchoring. The classifier's representation of the word will be near-random, and its predictions for sentences containing arabizi will be poor.
- **Remediations.**
  1. Use a multilingual tokenizer (e.g. `xlm-roberta-base`'s SentencePiece) trained on much broader data including Latin-script Arabic.
  2. Train a domain-specific tokenizer on a Tunisian-Arabic corpus before fine-tuning the model. The merges will then capture frequent dialectal sub-words natively.

2.1.9 Python snippet: comparing tokenizers

```python
# pip install nltk transformers
import nltk
from transformers import AutoTokenizer

text = "I've␣been␣to␣Sfax-University;␣it's␣amazing!"

# Word-level (NLTK)
nltk.download('punkt_tab', quiet=True)
print("Word␣tokens␣(NLTK):", nltk.word_tokenize(text))

# Sub-word (BERT WordPiece)
bert = AutoTokenizer.from_pretrained("bert-base-uncased")
print("BERT␣sub-word␣tokens:", bert.tokenize(text))

# Sub-word (GPT-2 byte-level BPE)
gpt2 = AutoTokenizer.from_pretrained("gpt2")
print("GPT-2␣BPE␣tokens:␣␣␣␣", gpt2.tokenize(text))
```

The same input produces different tokenisations under different algorithms. Reading the tokenized output is the first debugging step when a model behaves unexpectedly.

2.1.10 Exercises

1. For each scenario, recommend a tokenization granularity (word / character / sub-word) and justify briefly:
   (a) Building a spelling corrector for French SMS;
   (b) Training a sentiment classifier on 50 million Web reviews;
   (c) Building a name-entity recognizer for German legal documents.
2. Explain in one paragraph why character-level tokenization, despite its $|V| \approx 100$, has *not* replaced sub-word tokenization in modern LLMs.
3. Tokenize the sentence "*The CEOs' meeting was rescheduled.*" with two different policies (whitespace+punctuation, and a sub-word tokenizer of your choice). Comment on the differences for the words *CEOs'* and *rescheduled*.

4. Carry out three iterations of BPE on the toy corpus $\{\texttt{xay}, \texttt{xay}, \texttt{yax}, \texttt{xy}\}$ where each word has frequency 1. List the new vocabulary at each step.

5. (Synthesis.) For a multilingual Tunisian dialect dataset (mixing Arabic script, Latin-script arabizi, and French), discuss the trade-offs between (i) using a single multilingual sub-word tokenizer (e.g. XLM-R), (ii) training a domain-specific tokenizer on the dataset, and (iii) using language-specific tokenizers for each script and merging the outputs.